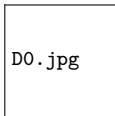


Lecture 16: Decomposition & 4NF

Handling Multivalued Facts and Ensuring Data Integrity

Z2004: Database Management Systems



IIT Madras Zanzibar

Thursday, 2 April 2026

Class Activity: Spot the Violation

The Scenario

A local clinic stores patient appointments in a single table:

Appointment

<u>ApptID</u>	PatID	PatName	DocID	DocName	Date
---------------	-------	---------	-------	---------	------

Functional Dependencies (FDs):

- 1 $\text{ApptID} \rightarrow \text{PatID}, \text{PatName}, \text{DocID}, \text{DocName}, \text{Date}$
- 2 $\text{PatID} \rightarrow \text{PatName}$
- 3 $\text{DocID} \rightarrow \text{DocName}$



Instructions

Take 5 minutes to solve the two tasks on the right.

Task 1: Identify the Anomaly

What specific anomaly occurs if Doctor "Dr. Smith" ($\text{DocID} = \text{D1}$) quits, and we delete all of his upcoming appointments from this table? Why?

Task 2: Normalize the Schema

1. What Normal Form is this table currently violating? (1NF, 2NF, 3NF, or BCNF?)
2. Decompose this single table into a set of visual tables that fully satisfy BCNF. Be sure to underline the Primary Keys.

Class Activity: Solutions

Task 1 Solution

Identify the Anomaly:

This is a **Deletion Anomaly**.

Why? If we delete the appointment rows to remove Dr. Smith's schedule, we simultaneously delete the PatName of the scheduled patients. We lose the record of those patients' names entirely from the clinic's database!

Task 2 Solution (Part 1)

What is the violation?

The table violates **3NF**. The Primary Key is ApptID. Both $\text{PatID} \rightarrow \text{PatName}$ and $\text{DocID} \rightarrow \text{DocName}$ are **transitive dependencies** (non-prime determining non-prime).

Task 2 Solution (Part 2)

The BCNF Decomposition:

We split out the transitive dependencies into their own tables. All left-hand sides of FDs are now superkeys!

Patient

<u>PatID</u>	PatName
--------------	---------

Doctor

<u>DocID</u>	DocName
--------------	---------

Appointment

<u>ApptID</u>	PatID	DocID	Date
---------------	-------	-------	------

1. Why BCNF isn't Enough?

The Recap

- ▶ **BCNF** eliminates all redundancy caused by *Functional Dependencies*.
- ▶ Rule: For every $X \rightarrow Y$, X must be a superkey.

The New Problem What if a table stores two *independent* many-to-many relationships?

Example: A Professor teaches multiple **Courses** AND has multiple **Research Interests**. These two facts are totally unrelated.

Table: Prof_Info (BCNF compliant! No FDs exist)

Prof	Course	Interest
Smith	DB	AI
Smith	OS	AI
Smith	DB	NLP
Smith	OS	NLP

Redundancy Alert: We are forced to repeat "AI" for every course Smith teaches. If Smith adds a new course, we must add multiple rows for every interest!

What is an MVD?

Two Independent Lists

A **Multivalued Dependency (MVD)** occurs when an entity has two independent lists of facts.

The Scenario: A Restaurant named "Bite".

- ▶ It serves: **Pizza** and **Burgers**.
- ▶ It delivers to: **North** and **South**.

Cuisines and Delivery Zones have *nothing* to do with each other. But look what happens when we put them in one table...

The "Cartesian" Explosion

Restaurant	Cuisine	Zone
Bite	Pizza	North
Bite	Pizza	South
Bite	Burgers	North
Bite	Burgers	South

⚠ The Nightmare:

If Bite adds "Tacos" to the menu, we cannot just add one row. We **MUST** add two rows (Tacos-North, Tacos-South) just to keep the data consistent!

2. Intuition: What is a Lossless Join?

The "Two Johns" Problem

When we decompose a table, we must be able to join it back together **perfectly**. If we split it poorly, joining creates fake data ("Spurious Tuples").

- ▶ **Lossless join:** Join gives *exactly* the original rows.
- ▶ **Lossy join:** Join creates *spurious* rows and/or loses rows.

Original Data

ID	Name	Dept
101	John	HR
102	John	IT

Let's split it into two tables:

- ▶ Table 1: (ID, Name)
- ▶ Table 2: (Name, Dept)

Rejoining on "Name"

The database sees "John" and matches him to every department John works in!

If a decomposition is **lossy**, you don't actually *lose* data; instead, you get **garbage** data (extra, fake rows) that weren't in the original table.

ID	Name	Dept	Status
101	John	HR	Real
101	John	IT	FAKE!
102	John	HR	FAKE!
102	John	IT	Real

💡 The Fix:

To avoid this, the column we split on **MUST** be a Unique ID (a Superkey) in at least one table!

3. Intuition: Dependency Preservation

Keeping Rules Together

A **Functional Dependency (FD)** is a business rule.

Example: $\text{ZipCode} \rightarrow \text{City}$.

Whenever a user inserts a row, the database checks to ensure they didn't put "New York" for a Los Angeles Zip Code.

A Poor Decomposition:

We have $\text{Table}(\text{Address}, \text{Zip}, \text{City})$.

We split it into:

- ▶ $R_1(\underline{\text{Address}}, \text{Zip})$
- ▶ $R_2(\text{Address}, \underline{\text{City}})$

The Rule is Broken!

Zip is in Table 1. City is in Table 2.

- ▶ If someone inserts a new Address, Zip, and City, how does the database check if $\text{Zip} \rightarrow \text{City}$ is valid?
- ▶ It has to pause, JOIN both tables together, run the check, and then save.
- ▶ Doing a JOIN on every single insert will destroy system performance!

The Goal: A good decomposition keeps the left and right sides of an FD inside the *same* table.

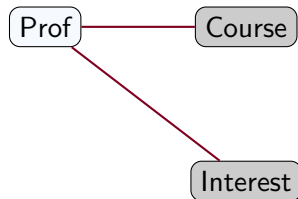
2. Multivalued Dependencies (MVDs)

The Definition An MVD $X \twoheadrightarrow Y$ exists if, for a given value of X , there is a set of values for Y that is **independent** of all other attributes in the table.

The Tuple-Switch Test:

If (x, y_1, z_1) and (x, y_2, z_2) are in the table, then (x, y_1, z_2) and (x, y_2, z_1) **must** also be in the table.

Visualizing the MVD



Notation: $\text{Prof} \twoheadrightarrow \text{Course}$
and $\text{Prof} \twoheadrightarrow \text{Interest}$.

3. Fourth Normal Form (4NF)

The Rule for 4NF A relation R is in 4NF if, for every non-trivial MVD $X \twoheadrightarrow Y$:

X is a superkey of R .

Why Prof_Info fails:

$\text{Prof} \twoheadrightarrow \text{Course}$ is an MVD, but **Prof** is not a superkey (one Prof has many courses/interests).

The 4NF Fix: Split! Decompose by creating a separate table for each independent multivalued fact.

Prof_Courses

<u>Prof</u>	<u>Course</u>
Otieno	DB
Otieno	OS

Prof_Interests

<u>Prof</u>	<u>Interest</u>
Otieno	AI
Otieno	NLP

4. Properties of Decomposition

When we split a table R into $R_1, R_2, \dots, R_n$, we must guarantee two things:

1. Lossless-Join

$$\Pi_{R_1}(R) \bowtie \Pi_{R_2}(R) = R$$

(Project R onto the attributes of R_1 and R_2 , then join; you should recover exactly R .)

If we join the tables back together, we must get the **exact** original data. No "spurious" (fake) rows allowed!

2. Dependency Preservation

$$(F_1 \cup F_2 \cup \dots \cup F_n)^+ = F^+$$

(Dependencies implied by checking each table's constraints are the same as those implied by the original set F .)

We should be able to enforce all original FDs by looking at each table individually, without needing to JOIN them first.

Important: BCNF always guarantees Lossless-Join, but NOT always Dependency Preservation!

5. Lossless Join: The "Spurious Tuple" Problem

If we decompose on an attribute that is **not a key** in at least one of the new tables, we lose information.

Original: Emp_Dept

EID	ENAME	Dept
101	Alice	Sales
102	Bob	Sales

Bad Split on (ENAME):

$R_1(\underline{\text{EID}}, \text{ENAME}), R_2(\text{ENAME}, \text{Dept})$

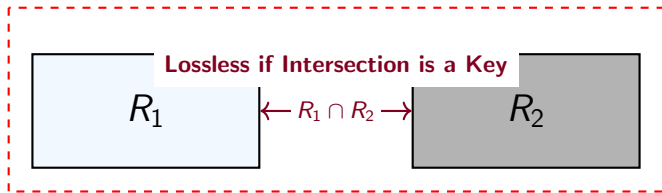
Result of $R_1 \bowtie R_2$:

EID	ENAME	Dept	Status
101	Alice	Sales	Original
102	Bob	Sales	Original
101	Bob	Sales	SPURIOUS!

Why? Because ENAME is not a key. Multiple people can have the same name, creating "fake" connections during the join.

6. Testing for Lossless Join (The Theorem)

A decomposition of R into (R_1, R_2) is lossless if and only if their common attributes $(R_1 \cap R_2)$ form a **superkey** for either R_1 or R_2 .



Example Check:

$R(A, B, C)$ with $A \rightarrow B$. Split into $R_1(A, B)$ and $R_2(A, C)$.

- ▶ Intersection = $\{A\}$.
- ▶ Is A a key for R_1 ? **Yes** (because $A \rightarrow B$).
- ▶ Conclusion: **Lossless Join guaranteed!**

💡 Note: When we reach 4NF, this rule expands slightly to accommodate Multivalued Dependencies (MVDs)!

7. Dependency Preservation

The Goal We want to check if a constraint like $A \rightarrow B$ still "lives" inside one of our new tables.

If A and B are separated into different tables, we cannot check $A \rightarrow B$ without performing an expensive JOIN every time someone inserts data.

The "Trade-off" Example $R(S, J, L)$ where S =Student, J =Job, L =Location.

FDs: $\{SJ \rightarrow L, L \rightarrow J\}$.

To get to BCNF, we *must* split:

① $R_1(\underline{L}, J)$

② $R_2(\underline{S}, L)$

Problem: $SJ \rightarrow L$ is LOST! It involves columns from both tables. This is a BCNF decomposition that is NOT dependency preserving.

Class Activity: 4NF and Lossless Join

Scenario A University database tracks **Courses**, the **Textbooks** required, and the **Rooms** where they are taught.

Table: Course_Info

Course	Book	Room
CS101	Book_A	Lab_1
CS101	Book_B	Lab_1
CS101	Book_A	Lab_2
CS101	Book_B	Lab_2

Fact: A course can have many books and many rooms. Books and Rooms are independent.

Tasks

- 1 Identify the MVDs in this table.
- 2 Is the table in 4NF? Explain.
- 3 Decompose the table into 4NF.
- 4 Prove your decomposition is **Lossless** using the intersection rule.

 **Time: 7 Minutes**

1 & 2. MVDs and 4NF

- ▶ **MVDs:** $\text{Course} \twoheadrightarrow \text{Book}$ and $\text{Course} \twoheadrightarrow \text{Room}$.
- ▶ **4NF? NO.** For $\text{Course} \twoheadrightarrow \text{Book}$ to be valid in 4NF, Course must be a superkey. It is not (CS101 appears 4 times).

3. Decomposition

Split into two tables:

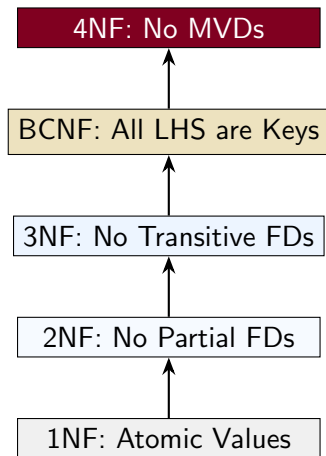
- ▶ **C_Books**(Course, Book)
- ▶ **C_Rooms**(Course, Room)

4. Lossless Proof

- ▶ $R_1 = \{\text{Course}, \text{Book}\}$, $R_2 = \{\text{Course}, \text{Room}\}$.
- ▶ $R_1 \cap R_2 = \{\text{Course}\}$.
- ▶ Is Course a key for R_1 or R_2 ?
- ▶ **Wait!** In 4NF theory, we use MVDs. The theorem states: Decomposition is lossless if the intersection **multidetermines** the difference.
- ▶ Since $\text{Course} \twoheadrightarrow \text{Book}$, the join is **Lossless**.

Logic: In 4NF, we don't need a superkey intersection; we need an MVD intersection!

Summary: The Normalization Ladder



NB: Most industry DBs stop at 3NF/BCNF. 4NF is essential for complex scientific or multi-fact catalogs.

What's Next? We have finished **Design Theory**. Now we move into how the database actually lives on hardware.

L17: Storage, Files & Buffer Pool

- ▶ Magnetic Disks vs. SSDs.
- ▶ How records are stored in "Pages".
- ▶ The Buffer Manager: RAM vs. Disk.

Homework

Decompose this schema to 4NF:

Restaurant(Name, Area, FoodType)

Rules:

1. A restaurant delivers to many areas.
2. A restaurant serves many types of food.
3. Areas and FoodTypes are independent.

Reading: Silberschatz, Ch 10 (Storage and File Structure).